TU Berlin
Institute of Mathematics
Professor Karow

TECHNISCHE UNIVERSITÄT BERLIN

Project in Numerical Analysis Summer 2026

Vortex Induced Vibration in Bladeless Turbine

Simulation and Analysis

Students: Hetvy Chanyiara, Petronella A. Kortleven, Nemanja Tesanoviç, Andrea Tomatis

1. Abstract

The global transition toward sustainable energy has prompted the development of novel wind harvesting systems. Among these, bladeless turbines represent a highly promising paradigm shift, utilizing flexible vertical masts to capture energy via vortex-induced aeroelastic resonance. This report presents a comprehensive computational study modeling a simplified 2D cantilever harvester using the Lattice Boltzmann Method. (add results)

2. Introduction

The EU has committed to becoming climate-neutral by 2050 by reducing fossil fuels and harnessing energy from renewable sources. Wind power plays a key role in this goal, requiring the installed capacity to expand rapidly toward 2030 targets (European Commission, Joint Research Centre 2025). Europe currently possesses 304 GW of total wind capacity, with the EU-27 accounting for 246 GW of that infrastructure (WindEurope 2026). However, scaling traditional rotating wind infrastructure faces severe restrictions due to several reasons. Firstly, a looming material sustainability crisis—with old, non-recyclable blades projected to generate up to 10 million tonnes of waste by 2050 (European Commission, Joint Research Centre 2025). Additionally, there is increased urban grid congestion and severe avian collision risks across Europe (WindEurope 2026; Etard, Jung, and Visconti 2026). Bladeless turbines directly eliminate these bottlenecks by substituting massive rotating blades with a compact, solid-state vertical mast that limits avian collision, is silent, and can be manufactured using recyclable thermoplastic materials.

Bladeless turbines operate entirely on the principle of vortex-induced vibration. When a fluid flows past a slender bluff body, the boundary layer separates due to adverse pressure gradients, leading to the periodic shedding of vortices from alternating sides of the structure. This phenomenon creates a predictable, highly structured wake pattern known in fluid dynamics as the Von Karman vortex street. The shedding of these vortices generates alternating low-pressure zones that exert a periodic lateral force on the structure perpendicular to the direction of the fluid flow. For maximum energy harvesting, the turbine structure must be carefully engineered such that its natural structural eigenfrequency matches the vortex shedding frequency dictated by the local wind speed. When these two frequencies align, the system enters a state of resonance known as lock-in. During the lock-in phase, the amplitude of the structural oscillation increases dramatically, allowing the internal linear alternators to convert the mechanical kinetic energy into usable electricity. If the wind speed drops too low, the shedding frequency cannot excite the structure. If the wind speed increases beyond the lock-in range, the shedding frequency outpaces the structure's physical capacity to respond, causing the vibration to dampen entirely.

In commercial deployments, an operational bladeless turbine consists of a rigid, three-dimensional tapered conical outer shell mounted atop an elastic internal carbon-fiber rod. This configuration acts as a rigid lever arm that tilts uniformly over a localized base pivot point (Vortex Bladeless 2026). Simulating this multi-body, 3D fluid-structure interface explicitly is computationally prohibitive for expansive parameter sweeps. Therefore, our framework simplifies the system into a two-dimensional side profile (X-Y plane) representing a homogeneous flexible column clamped at the base. To preserve the dynamics of the physical device within a 2D domain, a parabolic warping function is applied to the structural boundary mask. By scaling local horizontal grid displacements quadratically with height, the model ensures that structural mass distributions, elastic restoring forces, and ultimate kinetic power metrics accurately approximate the fundamental bending mode shape of a pivoting cantilever harvester.

Using this 2D framework, this project investigates how different environmental and mechanical variables fundamentally affect vortex shedding and structural resonance. Based on this approach, we aim to answer a central research question: **How do incoming fluid flows, material structural properties, and wake interactions dictate the energy harvesting efficiency of a bladeless wind turbine?**

To answer this systematically, we focus on three primary research goals:

1. Fluid Dynamics and Boundary Layer Impacts: We will analyze how changing the fluid flow across a critical Reynolds number range ($100 < Re < 1000$) influences vortex formation. This allows us to compare turbine performance under an idealized, uniform wind tunnel flow against a realistic natural wind profile that increases in speed with height.

2. Structural Tuning and Material Optimization: We will evaluate how the combination of structural mass, system damping, and bending stiffness (k) alters the turbine's movement. This helps pinpoint the exact material properties needed to achieve lock-in synchronization and maximize our power metric (P_{proxy}).

3. Wake Interaction and Array Placement: We will explore how these turbines behave when grouped together in close proximity. By placing multiple structures in a row, we can determine whether downstream turbines suffer from a loss of wind energy or if the wake from the lead turbine actually enhances their oscillations.

3. Theoretical Background

(a) Continuous Cantilever Beam Mechanics

The core energy-harvesting body of a bladeless wind turbine is an elastic vertical mast. Mechanically, this structure behaves as a continuous cantilever beam anchored rigidly at the ground boundary ($x = 0$) and completely free to deflect laterally at its upper tip ($x = L$). Because the structure is a continuous physical medium, it does not sway as a single rigid block. Instead, the horizontal deflection $y(x, t)$ varies significantly depending on the height coordinate x where you measure along the beam. A passing wind load will bend the upper sections of the mast far more aggressively than the stiff sections near the ground anchor.

According to Euler-Bernoulli beam theory, this continuous spatial deformation and its variation over time t are governed by a fourth-order partial differential equation:

$$EI\frac{\partial^4 y(x,t)}{\partial x^4} + m_L\frac{\partial^2 y(x,t)}{\partial t^2} + c\frac{\partial y(x,t)}{\partial t} = F_{\text{fluid}}(x,t) \tag{1}$$

In this formulation, EI represents the flexural rigidity of the material (where E is the material's Young's Modulus and I is the cross-sectional area moment of inertia), which acts as the structural restoring force trying to snap the bent beam back to center. The parameter m_L represents the

structural mass per unit length of the vertical column, which determines the physical inertia opposing local acceleration. The parameter c acts as the structural damping coefficient, modeling internal material friction that drains energy and opposes the velocity of the bending motion. The right-hand side, $F_{\text{fluid}}(x,t)$, represents the dynamic, distributed aerodynamic fluid load pressing against the cylinder at each specific height interval.

(b) Fluid Dynamics Framework

To calculate the aerodynamic forces acting on the mast, the surrounding air flow must be analyzed using the continuous Navier-Stokes equations. These equations describe the conservation of mass (continuity) and conservation of momentum for an incompressible, Newtonian fluid:

$$\nabla \cdot \mathbf{u} = 0 \tag{2}$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho_0}\nabla p + \nu \nabla^2 \mathbf{u} \tag{3}$$

where $\mathbf{u} = (u, v)$ represents the continuous macroscopic velocity field (the bulk measurable speed and direction of the wind), ρ_0 is the constant ambient fluid density, p is the static pressure field, and ν is the kinematic viscosity of the fluid. Solving these partial differential equations analytically in real-world engineering scenarios is exceptionally difficult due to the highly non-linear convective term $(\mathbf{u}\cdot\nabla)\mathbf{u}$, which models how the fluid's velocity field continuously accelerates itself.

To predict how the fluid behaves when interacting with our turbine without trying to solve these equations by hand, fluid dynamics relies on the dimensionless Reynolds number (Re):

$$Re = \frac{U_\infty L_{\text{char}}}{\nu} \tag{4}$$

where U_∞ represents the free-stream inflow velocity and L_{char} denotes the characteristic width profile of the turbine mast. The Reynolds number acts as the primary control dial for our wind tunnel environment. If Re is too low, viscous forces dominate, and the wind smoothly glides around the turbine without causing any movement. By tuning Re into a moderate laminar-to-turbulent transition window ($100 < Re < 1000$), we trigger the specific, alternating structural vibrations necessary to capture clean mechanical energy.

(c) Vortex-Induced Vibrations (VIV) and Lock-In Resonance

When the fluid flows past the bluff column inside our targeted Reynolds number window, the wind cannot handle the sharp pressure drops on the back curves of the cylinder. The boundary layer detaches from the solid surface and rolls up into spinning packets of low pressure called vortices. These vortices detach from alternating sides of the cylinder in a periodic cycle, trailing downstream to form a Von Kármán vortex street. The frequency f_s of this periodic aerodynamic shedding behavior is governed by the dimensionless Strouhal number (St):

$$St = \frac{f_s L_{\text{char}}}{U_\infty} \tag{5}$$

Because vortices are high-speed, spinning pockets of fluid, they create localized zones of low static pressure. When a vortex forms on the left side of the cylinder, it acts like a temporary vacuum that sucks the structure to the left. A moment later, that vortex sheds and moves downstream, and a new vortex forms on the right side, creating a vacuum that pulls the structure to the right. This alternating pressure difference on opposite sides of the mast creates a fluctuating asymmetric lift force perpendicular to the flow direction.

If the cantilever mast is flexible, this alternating lift force causes it to sway side-to-side. A critical synchronization phenomenon known as Lock-In occurs when the wind speed drives the fluid's vortex-shedding frequency f_s close to the structure's natural mechanical frequency f_n:

$$f_s \approx f_n \tag{6}$$

Within this Lock-In window, the physical movement of the swaying pole completely overrides the free-stream fluid dynamics. Instead of the wind dictating the vortex speed, the fluid is forced to shed its vortices at the structure's exact frequency of vibration. This eliminates phase lag, maximizes the transfer of kinetic energy from the wind into the structure, and results in massive, stable cross-flow oscillations.

(d) Aeroelastic Coupling and Power Extraction

The interplay between the fluid wake and the structural deflection forms a continuous feedback loop known as aeroelastic coupling. The fluid forces deform the structure, and the structure's altered position and velocity simultaneously modify the flow field, altering the vortex spacing and amplification.

Because the wind turbine captures energy linearly through this cross-flow sway rather than using rotating aerodynamic blades, it operates as a linear kinetic harvester. The instantaneous mechanical energy stored in this vibrating system is a combination of kinetic and potential energy. Assuming the structure achieves a steady-state harmonic oscillation defined by maximum amplitude A and operational frequency f, its tip position over time can be modeled as $\delta(t) = A\sin(2\pi f t)$. The instantaneous power extracted is proportional to the square of the structural velocity, which is the time derivative of its position. Integrating this velocity term over a complete steady-state harmonic cycle yields the average mechanical power capability:

$$P_{\text{avg}} = 2\pi^2 c A^2 f^2 \tag{7}$$

where c is the system damping coefficient. Because constant mechanical and structural coefficients can be isolated across comparative trials, we factor out these invariants to define the performance metric as $P_{\text{proxy}} \propto A^2 \cdot f^2$, where A represents the maximum deflection amplitude and f is the operational frequency.

Optimizing power extraction requires managing a fundamental engineering trade-off. A material that is excessively stiff will yield high frequencies but restricted deflection amplitudes, while a material that is overly flexible will yield massive amplitudes but slow, inefficient frequencies. Maximizing the power proxy relies entirely on tuning the material to achieve peak aeroelastic coupling within the Lock-In regime.

4. Methodology and Numerical Implementation

(a) Structural Simplification

Simulating a continuous fourth-order partial differential equation simultaneously with a fluid solver is computationally expensive. To optimize performance during the fluid-structure interaction sequence, a lumped-parameter approximation is applied. The continuous beam is mathematically reduced to a single point mass m supported by a theoretical spring with stiffness k and a damper with coefficient c. This transforms the complex partial differential equation into a solvable second-order ordinary differential equation governing the horizontal tip deflection δ:

$$m\ddot{\delta} + c\dot{\delta} + k\delta = F_{\text{fluid}}(t) \tag{8}$$

To bridge the structural model with the fluid grid, we map the structural coordinate system onto the 2D Cartesian simulation space. While classical beam theory treats the height of the

column as the x-axis, the 2D fluid lattice grid defines the horizontal flow direction as x and the vertical altitude as y. Thus, the physical mast is anchored at the grid base and spans vertically along the grid's y-coordinate.

To ensure this ordinary differential equation accurately reflects a real macroscopic turbine of height L, width w, and wall thickness t_{wall}, the equivalent stiffness k is explicitly derived from Euler-Bernoulli theory for a tip-loaded cantilever. The area moment of inertia I is calculated as the structural width multiplied by the cubed thickness divided by twelve, yielding the final stiffness relation:

$$k = \frac{3EI}{L^3} \tag{9}$$

The net dynamic fluid force $F_{\text{fluid}}(t)$ driving this motion is computed by integrating the pressure variances across the front and rear faces of the discrete object mask grid:

$$F_{\text{fluid}}(t) = \sum_{\mathbf{x} \in \text{Upstream}} \rho(\mathbf{x}, t) - \sum_{\mathbf{x} \in \text{Downstream}} \rho(\mathbf{x}, t) \tag{10}$$

To step this mechanical system forward in time alongside the fluid iterations, the software solves the system using a discrete explicit Euler integration scheme. For each discrete time step $\Delta t = 1$, the updates are computed sequentially:

$$a(t) = \frac{F_{\text{fluid}}(t) - c \cdot v(t) - k \cdot \delta(t)}{m} \tag{11}$$

$$v(t+1) = v(t) + a(t) \cdot \Delta t \tag{12}$$

$$\delta(t+1) = \delta(t) + v(t+1) \cdot \Delta t \tag{13}$$

(b) The Lattice Boltzmann Method (D2Q9 LBM Framework)

Traditional computational fluid dynamics packages rely heavily on the Finite Volume Method (FVM) or Finite Element Method (FEM) to solve the Navier-Stokes equations. While highly accurate, these continuum methods are computationally intensive because they require solving massive global matrix systems at every time step to link pressure and velocity together. Furthermore, when applied to moving structures, FVM and FEM suffer from severe mesh complications; the fluid mesh must physically warp, stretch, and re-mesh at every step to follow the turbine's moving surface, which easily leads to inverted grid volumes and simulation crashes.

To bypass these geometric bottlenecks, the Lattice Boltzmann Method (LBM) is widely used in the field of fluid-structure interaction. Instead of moving the grid, LBM utilizes a frozen, fixed Cartesian pixel map. Solid structural objects are defined directly within this spatial grid using a boolean mask matrix $M(\mathbf{x}) \in \{0, 1\}$, where a cell value of 1 designates a solid node and 0 represents open fluid. This allows the moving turbine to be tracked cleanly across a static grid, letting the pole bend freely without requiring any mesh deformation or grid regeneration.

Our code implements the popular D2Q9 framework, a two-dimensional spatial discretization that restricts particle movement to nine discrete lattice velocity vectors ($\mathbf{v}_i$). Rather than tracking continuous trajectories, the fluid molecules are idealized as probability distribution functions that are constrained to hop along these nine distinct routes at each time step. These velocity vectors match the directional offsets defined in the codebase:

$$\mathbf{v}_i = \Big[[1,1], \quad [1,0], \quad [1,-1], \quad [0,1], \quad [0,0], \quad [0,-1], \quad [-1,1], \quad [-1,0], \quad [-1,-1]\Big] \tag{14}$$

Geometrically, this specific arrangement represents a central stationary point and eight surrounding discrete directions. Indexing choices in code often vary, but the vector collection cleanly

groups into three distinct categories: one static center position where particles do not move ($[0, 0]$), four orthogonal steps pointing directly along the principal coordinate axes ($[1, 0]$, $[0, 1]$, $[-1, 0]$, and $[0, -1]$), and four diagonal directional steps connecting to the corner points ($[1, 1]$, $[1, -1]$, $[-1, 1]$, and $[-1, -1]$).

To maintain spatial isotropy so that fluid flows identically in all directions across the pixel grid, each velocity direction is scaled by a fixed geometric weight matrix:

$$t_i = \left[\frac{1}{36}, \quad \frac{1}{9}, \quad \frac{1}{36}, \quad \frac{1}{9}, \quad \frac{4}{9}, \quad \frac{1}{9}, \quad \frac{1}{36}, \quad \frac{1}{9}, \quad \frac{1}{36}\right] \tag{15}$$

These lattice weights (t_i) act as geometric balancing factors that ensure the discrete lattice behaves exactly like a continuous, isotropic fluid. They correspond directly to the breakdown of our velocity vectors: the stationary center channel holds the highest local weight configuration at $4/9$, the four main orthogonal axes are distributed evenly with weights of $1/9$ each, and the four longer diagonal transitions are scaled down to weights of $1/36$ each to compensate for their greater geometric lengths.

The microscopic streaming and collision actions of these particles are governed by the discrete Bhatnagar-Gross-Krook (BGK) equation:

$$f_i(\mathbf{x} + \mathbf{v}_i, t + 1) = f_i(\mathbf{x}, t) - \frac{1}{\tau}\left[f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)\right] \tag{16}$$

where τ is the relaxation time parameter, which controls how fast the virtual fluid molecules crash and relax toward an equilibrium state. This single-relaxation equation operates as a two-part loop that runs at every node across the simulation lattice. During the collision phase, the current particle distributions (f_i) are relaxed toward an ideal local thermodynamic equilibrium (f_i^{eq}) based on the relaxation rate $\omega = 1/\tau$, which models how internal collisions distribute momentum. Immediately following this relaxation, the streaming step takes over, shifting each updated distribution function linearly to its neighboring lattice node along its chosen velocity vector direction $\mathbf{v}_i$.

In the code, the local equilibrium distribution function f_i^{eq} is calculated algebraically at every single pixel using a low-Mach-number Taylor expansion based on the local velocity and density:

$$f_i^{\text{eq}}(\mathbf{x}, t) = \rho t_i \left[1 + 3(\mathbf{v}_i \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{v}_i \cdot \mathbf{u})^2 - \frac{3}{2}(\mathbf{u} \cdot \mathbf{u})\right] \tag{17}$$

To bridge these virtual particle representations back into observable, real-world properties, the macroscopic fluid density ρ and macroscopic velocity $\mathbf{u}$ are recovered directly by calculating the local sums of the distribution functions at that node:

$$\rho = \sum_{i=0}^{8} f_i, \quad \rho\mathbf{u} = \sum_{i=0}^{8} f_i \mathbf{v}_i \tag{18}$$

Evaluating these macroscopic variables is equivalent to taking statistical moments of the probability density distributions. The zeroth moment computes the local fluid density (ρ) simply by summing all nine components together, while the first moment yields the local momentum ($\rho\mathbf{u}$) by taking the weighted sum of the distributions multiplied by their directional vector values.

Dividing the resulting momentum components by the density isolates the actual velocity field vector $\mathbf{u} = (u, v)$ at that point. Executing these simple, local particle hopping and bouncing steps perfectly recreates the complex continuous macroscopic Navier-Stokes fluid equations.

(c) Boundary Interactions and Environmental Velocity Profiles

Interaction at the solid-fluid interface is resolved using a no-slip bounce-back boundary scheme. When a distribution function streams into a node designated as solid by our boolean mask ($M(\mathbf{x}) = 1$), it is reflected directly back toward its originating node:

$$f_i(\mathbf{x}, t+1) = f_{\bar{i}}(\mathbf{x}, t), \quad \text{where } \mathbf{v}_{\bar{i}} = -\mathbf{v}_i \tag{19}$$

While standard wind tunnel configurations impose an idealized, uniform inlet flow profile where velocity is constant across all heights ($u_{\text{inlet}}(y) = U_\infty$), our numerical pipeline simulates realistic atmospheric boundary layers using a custom wind shear condition. This vertical gradient is enforced at the fluid inlet across the vertical grid space y:

$$u_{\text{inlet}}(y) = U_{\text{nominal}} \cdot \left[0.20 + 1.60 \cdot \left(\frac{y}{N_y}\right)\right] \tag{20}$$

where U_{nominal} represents the target design speed and N_y is the maximum vertical grid dimension.

This profile forces the wind speed at ground level ($y = 0$) to be just 20% of nominal design speeds to simulate surface friction, while scaling it up linearly to 180% at the top edge ($y = N_y$). This introduces a strong velocity gradient $\frac{\partial u}{\partial y}$ across the vertical profile of the cantilever mast. Introducing this choice allows us to evaluate whether the turbine can maintain stable lock-in synchronization when different heights of the structure experience different wind forces, directly contrasting the clean single-frequency responses of a uniform wind tunnel with the multi-frequency interactions of a real boundary layer.

5. Methods and Implementation Framework

The overarching computational framework is engineered using a highly modular, object-oriented architecture written in Python. To facilitate automated batch testing, parameter sweeps, and strict separation of concerns, the codebase is partitioned into five primary modules. This segregation ensures that the fluid physics, spatial geometry, simulation orchestration, test generation, and data analysis operate independently, allowing for complex multi-body interactions without requiring modifications to the underlying numerical engines.

(a) Simulation Configuration and Execution (`main.py`)

The entry point of the framework is responsible for handling the global simulation state and orchestrating the temporal loop. To guarantee reproducibility across experiments, all physical and environmental parameters are encapsulated within a strongly typed data structure. This configuration object acts as the single source of truth for the lattice dimensions, Reynolds number, characteristic length, and output frequencies. It also dynamically computes derived lattice properties, such as the kinematic viscosity and the corresponding collision relaxation frequency.

```
@dataclass
class SimConfig:
    nx: int = 600
    ny: int = 200
    Re: float = 250.0
    uLB: float = 0.04
    max_iter: int = 5000
    L_char: float = 20.0
    top_bottom_walls: bool = False
    flow_profile: str = 'uniform'

    @property
    def nulb(self):
```

```
        return self.uLB * self.L_char / self.Re

    @property
    def omega(self):
        return 1.0 / (3.0 * self.nulb + 0.5)
```

The primary execution loop iteratively advances the fluid solver and processes the resulting data. At specified intervals, the loop extracts the maximum fluid velocity and the structural deflection of every object present in the scene, appending this data to a comma-separated values log. Concurrently, it computes the fluid vorticity curl and maps it to a color matrix, rendering the physical state into a compressed video file via the OpenCV library.

(b) Fluid-Structure Interaction Geometry (`geometry.py`)

The geometric module dictates how solid boundaries are defined and updated within the discrete lattice. A master scene container aggregates multiple arbitrary geometric objects, applying a logical OR operation across their respective boolean masks to generate a unified obstacle map. The most critical component of this module is the cantilever class, which models the bladeless turbine.

At each time step, the turbine reads the local macroscopic fluid density directly from the solver. By sampling the lattice cells immediately adjacent to its upstream and downstream faces, it calculates the pressure differential. Because Lattice Boltzmann pressure is linearly proportional to density, this difference multiplied by a stabilization scaling factor yields the net aerodynamic fluid force.

```
# Inside FlexibleCantilever.get_mask()
up_x = max(0, int(self.cx - self.w))
down_x = min(rho.shape[0] - 1, int(self.cx + self.w + self.delta))
y_range = slice(int(self.cy_base), int(self.cy_base + self.h))

p_up = np.sum(rho[up_x, y_range]) / 3.0
p_down = np.sum(rho[down_x, y_range]) / 3.0
F_fluid = (p_up - p_down) * 0.01
```

This continuous fluid force is immediately passed into an explicit Euler numerical integrator. The integrator solves the second-order ordinary differential equation governing the lumped-parameter mass-spring-damper system. To maintain numerical stability and prevent mathematical explosions during resonance, strict clipping limits are enforced on both the velocity and the ultimate deflection.

```
acceleration = (F_fluid - self.k * self.delta - self.c * self.vel) / self.m
self.vel += acceleration
self.delta += self.vel

if self.delta > self.h/2:
    self.delta = self.h/2
    self.vel = 0.0
elif self.delta < -self.h/2:
    self.delta = -self.h/2
    self.vel = 0.0
```

Once the scalar deflection of the mast tip is computed, the object must update its physical footprint within the two-dimensional grid. To accurately simulate the bending of a continuous cantilever beam subjected to a distributed load, the geometric boolean mask is warped. The horizontal shift of any given vertical point follows a parabolic curve, normalized against the total height of the structure.

```
Y_norm = np.clip((Y - self.cy_base) / self.h, 0, 1)
```

```
dX = self.delta * (Y_norm ** 2)

y_bounds = (Y >= self.cy_base) & (Y <= self.cy_base + self.h)
x_bounds = np.abs(X - (self.cx + dX)) <= self.w / 2

self.last_mask = y_bounds & x_bounds
```

(c) Fluid Dynamics Engine (`solver.py`)

The solver module encapsulates the Lattice Boltzmann Method algorithms. While the mathematical theory of the D2Q9 lattice and the Bhatnagar-Gross-Krook collision operator were detailed in the preceding section, the software implementation relies heavily on multi-dimensional array vectorization. Instead of iterating through individual grid cells using nested loops, the solver leverages NumPy element-wise operations to compute the macroscopic variables and the collision relaxations simultaneously across the entire domain.

Within the primary solver step, the temporal sequence is strictly ordered: macroscopic variable recovery is followed by inflow condition enforcement, equilibrium calculation, collision relaxation, boundary condition application, and finally, spatial streaming. The dynamic obstacle mask generated by the geometry module is integrated directly after the collision step, applying the bounce-back reflection condition specifically to the cells currently occupied by the moving turbine.

```
# Collision step
fout = self.fin - self.cfg.omega * (self.fin - feq)

# Fetch the dynamic geometry mask
obstacle = self.scene.get_mask(self.X, self.Y, iter_step, rho, u)

# Apply bounce-back to the moving obstacle
for i in range(9):
    fout[i, obstacle] = self.fin[8-i, obstacle]
```

(d) Automated Testing and Batch Processing (`test.py`)

To conduct rigorous scientific parameter sweeps without manual intervention, the testing module programmatically generates JavaScript Object Notation configuration files. This automated suite orchestrates the execution of four distinct experimental phases, each designed to isolate and evaluate specific physical variables governing bladeless turbine performance.

(d).1 Phase 1: Aeroelastic Lock-in Sweep

The primary objective of the first testing phase is to empirically identify the exact lock-in resonance curve for a baseline turbine geometry. To achieve this, the test suite generates a batch of simulations where the structural properties, specifically the mass and stiffness, are held strictly constant. The simulation utilizes stable mathematical parameters, setting the structural stiffness to 0.008, the mass to 3.0, and the damping coefficient to 0.002. The overarching loop systematically increments the fluid Reynolds number across five distinct steps: 150, 200, 250, 300, and 350.

```
# Systematic parameter sweep to identify the resonance peak
simulations = []
for re in [150, 200, 250, 300, 350]:
    name = f"P1_Re_{re}"
    simulations.append({
        "name": name,
        "re": float(re),
        "flow": "uniform",
        # Structural parameters remain constant across all iterations
```

```
        "objects": [{"shape": "flexible_pole", "stiffness": 0.008,
                     "mass": 3.0, "damping": 0.002}]
    })
```

By generating discrete outputs for each velocity step spanning this range, the phase ensures the fluid's Strouhal shedding frequency crosses the structure's natural eigenfrequency. The inclusion of the damping parameter prevents infinite amplitude blow-up at the exact lock-in threshold, allowing the analyzer to capture the exponential rise and stable plateau of the harmonic oscillation amplitude.

(d).2 Phase 2: Material Optimization Matrix

Once the optimal lock-in Reynolds number is established (fixed at 250.0 for this phase), the second module evaluates the structural domain. A critical engineering function of this module is non-dimensionalization. Raw macroscopic material properties cannot be directly ingested by the explicit fluid-structure interaction solver without causing catastrophic numerical instability. Therefore, the testing module defines a matrix of four real-world materials: polyvinyl chloride plastic, fiberglass, aluminum, and carbon fiber.

The routine calculates the true area moment of inertia and physical stiffness for an eight-meter-tall mast based on the specific Young's Modulus and density of each material. For instance, it accounts for the massive rigidity difference between carbon fiber (150 gigapascals) and PVC plastic (3 gigapascals). It then multiplies these real-world values by constant scaling factors, specifically 10^{-6} for stiffness and 10^{-5} for mass.

```
# Real-world material properties mapped to stable LBM units
materials = {
    "PVC_Plastic":  {"E": 3.0e9,   "rho": 1380},
    "Fiberglass":   {"E": 15.0e9,  "rho": 1900},
    "Aluminum":     {"E": 69.0e9,  "rho": 2700},
    "Carbon_Fiber": {"E": 150.0e9, "rho": 1600}
}

scale_k = 1e-6
scale_m = 1e-5

for mat_name, props in materials.items():
    real_k = (3 * props["E"] * I) / (height_m ** 3)
    lbm_stiffness = real_k * scale_k
    lbm_mass = (props["rho"] * volume) * scale_m
```

This scaling methodology strictly preserves the relative density and elasticity ratios between the materials, allowing the solver to accurately simulate how a heavily rigid material behaves under identical aerodynamic loads compared to a highly elastic material without shattering the discrete time integration.

(d).3 Phase 3: Atmospheric Boundary Layer Impact

The third experimental phase transitions the testing environment from an idealized wind tunnel to a realistic atmospheric scenario. Real-world wind does not strike vertical structures uniformly; it creates a shear boundary layer where velocity increases continuously with altitude. The automated script iterates over a binary list to toggle the computational domain's inflow condition between a uniform profile and a shear gradient.

```
# Toggling the inflow boundary conditions for environmental testing
for flow_type in ['uniform', 'shear']:
    name = f"P3_Flow_{flow_type.capitalize()}"
```

```
    simulations.append({
        "name": name,
        "flow": flow_type,
        "objects": [{"shape": "flexible_pole", "cx": 200, "cy": 1,
                     "width": 10, "height": 100,
                     "stiffness": 0.005, "mass": 2.0}]
    })
```

Both simulations evaluate an identical generic flexible mast over 8000 computational steps. This comparative test is critical for quantifying the destructive interference caused by shear flow, demonstrating how variations in local shedding frequencies along the vertical axis of the mast degrade the overall energy harvesting efficiency.

(d).4 Phase 4: Multi-Turbine Wake Interference

The final testing phase scales the simulation from a single isolated mast to a high-density wind farm layout. To evaluate aerodynamic synergy and wake-induced flutter, the configuration generator places four independent flexible cantilevers into an in-line tandem array. To mimic actual operational conditions, this phase exclusively utilizes the shear flow boundary condition.

```
# Generating a highly dense multi-mast array layout
"objects": [
    {"shape": "flexible_pole", "cx": 200, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 450, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 700, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 950, "stiffness": 0.006, "mass": 2.5}
]
```

The turbines are uniformly spaced 250 spatial units apart along the longitudinal axis, occupying a significantly extended computational domain of 1100 by 300 cells. Because the scene geometry module dynamically aggregates all object masks, the underlying Lattice Boltzmann solver naturally resolves the complex fluid mechanics of the upstream vortices impacting the downstream structures. This layout enables the data analyzer to directly compare the oscillation phases and power yields of the wake-embedded turbines against the primary upstream mast over an extended duration of 12000 steps.

(e) Data Extraction and Performance Analysis (`analysis.py`)

The final module functions as the post-processing pipeline, converting raw discrete time-series data into continuous performance metrics and publication-ready visualizations. Utilizing the Pandas library, the analyzer ingests the structural logs generated by the batch tests. To ensure accurate steady-state harmonic analysis, the script isolates a specific analytical window, explicitly ignoring the transient fluid spin-up and initial structural startup phases.

To evaluate the comparative efficiency of different turbine configurations, the analyzer extracts both the maximum continuous amplitude and the predominant oscillation frequency. The frequency is computed algorithmically by identifying the indices where the derivative of the positional sign changes, which precisely isolates the structural zero-crossings across the neutral axis. Half the number of these zero-crossings divided by the elapsed simulation steps yields the normalized frequency.

```
# Isolate steady-state oscillation
plot_data = df[(df['Step'] > 2000) & (df['Step'] < 8000)]

# Calculate Amplitude (Mean of absolute peaks)
amplitude = plot_data['Deflection_dX'].abs().mean()

# Calculate Frequency via zero-crossings
```

```
zero_crossings = np.where(np.diff(np.sign(plot_data['Deflection_dX'])))[0]
num_cycles = len(zero_crossings) / 2.0
time_steps = plot_data['Step'].max() - plot_data['Step'].min()
frequency = num_cycles / time_steps

# Compute theoretical mechanical energy
power_proxy = (amplitude ** 2) * (frequency ** 2)
```

These extracted values are then fed into the normalized mechanical power proxy equation discussed in the introduction. The module dynamically generates comparative line plots of the positional waveforms, evaluates wake-induced efficiency ratios for multi-mast arrays, and exports a fully ranked numerical leaderboard of material performance to a secondary formatted dataset.

6. Results

TODO.

(a) Conclusions

TODO.

References

European Commission, Joint Research Centre (2025). *Circular Economy Strategies for the EU's Renewable Electricity Supply*. CEPRES Exploratory Research Project Report. Luxembourg: Publications Office of the European Union. URL: https://publications.jrc.ec.europa.eu/repository/handle/JRC138570.

WindEurope (Feb. 2026). *Wind energy in Europe: 2025 Statistics and the outlook for 2026-2030*. Brussels, Belgium: WindEurope. URL: https://windeurope.org/data/product/wind-energy-in-europe-2025-statistics-and-the-outlook-for-2026-2030/.

Etard, A., M. Jung, and P. Visconti (2026). "Risk to European birds from collisions with wind-energy facilities". In: *Biological Conservation* 319, e111875. DOI: 10.1016/j.biocon.2026.111875. URL: https://doi.org/10.1016/j.biocon.2026.111875.

Vortex Bladeless (2026). *Vortex Bladeless Wind Energy: Vortex-Induced Vibration Aerogenerators*. Official Company Website. URL: https://vortexbladeless.com/.